

Name: Narendra Kumar S

Date : 18-08-2025

Task:

create fast api to store and retrieve specific song by the id

sqlite can be used as data store

table can contain song name playing time in the seconds album along with id

Solution:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker, Session, declarative_base
from fastapi import FastAPI, Depends, Request, HTTPException
from typing import List
from pydantic import BaseModel
import uvicorn

SQLALCHEMY_DATABASE_URL = "sqlite:///C:\\Users\\narendrakumar.s\\songs.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL,
connect_args={"check_same_thread": False})

SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
Base = declarative_base()

class SongModel(Base):
    __tablename__ = "songs"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    playtime = Column(Integer, nullable=False)
    album = Column(String(100), nullable=False)

Base.metadata.create_all(bind=engine)

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
```

```

        yield db
    finally:
        db.close()

class Song(BaseModel):
    id: int
    name: str
    playtime: int
    album: str
    class Config:
        orm_mode = True

class SongDTO(BaseModel):
    name: str
    class Config:
        orm_mode = True

@app.post("/add_song", response_model=Song)
def add_song(song: Song, db: Session = Depends(get_db)):
    new_song = SongModel(name=song.name, playtime=song.playtime,
album=song.album)
    db.add(new_song)
    db.commit()
    db.refresh(new_song)
    return new_song

@app.get("/list_songs", response_model=List[Song])
def list_songs(db: Session = Depends(get_db)):
    return db.query(SongModel).all()

@app.get("/get_song/{song_id}")
async def get_song(song_id: int, db: Session = Depends(get_db)):
    song = db.query(SongModel).filter_by(id=song_id).first()
    st_dao = SongDTO(name=song.name)
    return st_dao

if __name__ == "__main__":
    uvicorn.run("Songs:app", host="127.0.0.1", port=8000, reload=True)

```

Execution:

```
WARNING: StatReload detected changes in 'Songs.py'. Reloading...
D:\python\Task\Task\Lib\site-packages\pydantic\_internal\_config.py:373: UserWarning: Valid config keys have changed in V2:
* 'orm_mode' has been renamed to 'from_attributes'
  warnings.warn(message, UserWarning)
INFO: Started server process [12068]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Output:

The screenshot shows a web browser window with a REST client interface. The URL bar shows `127.0.0.1:8000/docs#/default/add_song_add_song_post`. The interface is for a **POST** request to `/add_song` with the description "Add Song".

Under the **Parameters** tab, it says "No parameters". There are "Cancel" and "Reset" buttons.

Under the **Request body** tab, which is marked as "required", there is a dropdown menu set to `application/json`. Below this, there are tabs for "Edit Value" and "Schema". The "Edit Value" tab is active, showing a JSON object in a text area:

```
{
  "id": 7,
  "name": "Ratha mara",
  "playtime": 169,
  "album": "Jailer"
}
```

At the bottom of the interface, there are two buttons: "Execute" (in blue) and "Clear".

127.0.0.1:8000/docs#/default/add_song_add_song_post

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/add_song' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 7,
    "name": "Ratha mara",
    "playtime": 169,
    "album": "Jailer"
  }'
```

Request URL

http://127.0.0.1:8000/add_song

Server response

Code Details

200

Response body

```
{
  "id": 7,
  "name": "Ratha mara",
  "playtime": 169,
  "album": "Jailer"
}
```

Response headers

```
content-length: 60
content-type: application/json
date: Mon, 18 Aug 2025 12:09:09 GMT
server: uvicorn
```

Responses

Code Description Links

DB Browser for SQLite - C:\Users\narendrakumar.s\songs.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

songs songs

Table: songs

	id	name	playtime	album
Filter	Filter	Filter	Filter	Filter
1	6	Vathi coiming	230	Mas...
2	7	Ratha mara	169	Jai...
3	5	Kanima	167	Ret...

1 3 of 3

Go to: 1

Create a new database file

Edit Database Cell

Mode: Text

1

Editing row=1, column=0
Type: Text / Numeric; Size: 1 character(s)

Apply

Remote

Identity Select an identity to connect Upload

DBHub.io Local Current Database

Name Last modified Size

SQL Log Plot DB Schema Remote

List of All Song in DataBase

Request URL

`http://127.0.0.1:8000/list_songs`

Server response

Code	Details
200	Response body <pre>[{"id": 5, "name": "Kanima", "playtime": 167, "album": "Retro"}, {"id": 6, "name": "Vathi coiming", "playtime": 230, "album": "Master"}, {"id": 7, "name": "Ratha mara", "playtime": 169, "album": "Jailer"}, {"id": 8, "name": "Ratha mara", "playtime": 169, "album": "Jailer"}]</pre> Response headers <pre>content-length: 243 content-type: application/json date: Mon, 10 Jul 2023 13:48:31 GMT</pre>

Download

Get Song By ID

GET /get_song/{song_id} Get Song

Parameters

Name	Description
song_id * required	
integer	
(path)	

7

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/get_song/7' \
-H 'accept: application/json'
```

Request URL

`http://127.0.0.1:8000/get_song/7`

127.0.0.1:8000/docs#/default/get_song_get_song_id_get

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/get_song/7' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/get_song/7
```

Server response

Code	Details
200	Response body <pre>{ "name": "Ratha mara" }</pre> Download Response headers <pre>content-length: 21 content-type: application/json date: Mon, 18 Aug 2025 12:11:30 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links